

Recommendations Overview

The Collaborative Knowledge Hub specification is a strong start. The project has a useful direction: a reliable knowledge and membership system for ASC that can later support graph-based discovery, member matching, learning journeys, and social forms of collaboration.

You asked me to critique the technology stack. My assessment is that the proposed stack is reasonable for the expected scale, but several choices are asserted rather than designed, justified, or compared. React/Next.js, FastAPI, PostgreSQL, Docker Compose, Nginx, Stripe, Google Workspace integration, and external AI APIs are all plausible. The concern is that the document does not yet show its reasoning.

For example, the document assumes a self-managed Linux server or VPS without explaining why that's preferable to managed hosting, managed PostgreSQL, serverless, or platform-as-a-service. For a small volunteer-run organization, operational simplicity often matters more than the lowest monthly bill. The deployment model deserves an explicit evaluation against maintenance burden, security updates, backup and restore, monitoring, cost predictability, data residency, and handover complexity.

At ASC's scale, PostgreSQL is sufficient for the graph model the team has described; a separate graph database is not necessary unless the team can identify graph queries PostgreSQL cannot reasonably support. But the document should still clarify whether the graph will be modeled relationally, generated as a projection, or exported for external analysis.

The same applies to AI. Using an external provider may be reasonable, but the document needs to define the agent architecture: system instructions, prompt templates, model choices, parameters, knowledge bases, permissions, logging, traceability, evaluation, and human review. Without that, "AI" and "agentic" remain feature labels rather than designed parts of the system.

In short: the proposed technologies are plausible, but a stack critique can't stop at naming tools. The important questions are whether the architecture is maintainable by volunteers, supports future graph and social features without overbuilding the MVP, protects international member data, and governs AI safely. The recommendations below address those concerns.

Some of the suggestions below are intentionally concrete. They are not meant to prescribe final implementation choices or reduce your design freedom. They are examples of the level of specificity I think the project needs: clearer workflows, explicit design decisions, testable requirements, and documented tradeoffs. Where I suggest a possible approach, please treat it as a starting point for design discussion rather than as a fixed instruction.

I recommend the following revisions:

1. Make the volunteer admin portal a core MVP feature.
2. Define the MVP as graph-ready, not graph-complete.
3. Add a design document covering user journeys, admin workflows, data model, and AI architecture.
4. Maintain a decision log capturing design intent, tradeoffs, and architectural choices.
5. Add an international data policy and data governance model.
6. Add a community safety, abuse handling, and moderation model.
7. Replace vague agentic claims with a concrete agentic-system design.
8. Prioritize an Administration Support Agent over an external scraper.
9. Require human review for AI outputs affecting members, public content, permissions, payments, communications, or moderation outcomes.
10. Clarify the MVP boundary.
11. Rework learning journeys as either deferred features or simpler curated collections.
12. Treat knowledge graph design and member matching as two separate design problems.
13. Add testable acceptance criteria for admin operations, graph-readiness, import/export, community safety, AI behavior, matching, and graph visualization.
14. Restructure the MVP and extension priority list.

Recommendations for Strengthening the Collaborative Knowledge Hub Specification and Design: **Overview**

Thomas J McLeish, May 23, 2026

These recommendations share a theme: the MVP should be a reliable, volunteer-operable foundation for future graph, social, and AI features. The first design priority is not only what members see, but how the organization will run the system safely over time.

Happy to discuss any of this if useful.

tjm*

* I have used the source documents provided, my own experience in building out similar things, and collaboration with my ai assistants in preparing this review. I have read, edited, and approved this document.

Brief Follow Up

One brief follow-up to my earlier note.

After thinking more about the proposed stack, I want to sharpen one point: I am not sure a separate FastAPI/Python backend makes sense for ASC's long-term production needs unless there is a specific reason for it.

FastAPI may be a reasonable prototype choice if the team is comfortable with Python. Python may also be useful later for specialized graph analysis, AI workflows, or data-processing jobs. However, those needs do not automatically require a separate Python backend for the MVP. Many of the likely MVP tasks can be handled by Next.js/TypeScript, PostgreSQL, managed services, scheduled jobs, or external analysis tools. A small Python worker or service could be added later if a specific graph, AI, or data-processing requirement genuinely calls for it.

For the MVP as described, much of the application appears to be membership management, role-based administration, knowledge hub workflows, moderation, tags, search, import/export, and integrations. Those needs could plausibly be handled in a single Next.js/TypeScript application with PostgreSQL, especially if the goal is to reduce operational complexity for a small volunteer-run organization.

TypeScript is also relevant here. A Next.js/TypeScript architecture gives the application explicit types, interfaces, schemas, and compile-time checks for users, resources, roles, permissions, submissions, tags, relationships, agent outputs, and API responses. That helps human developers, and it also helps AI-assisted development because the codebase carries more machine-readable structure. For a project that may be maintained or extended later with AI assistance, a strongly typed TypeScript codebase can make workflows, API contracts, data models, and permissions easier to inspect, refactor, and verify.

This is especially relevant if graph visualization will be implemented with Three.js and SVG. Those tools live naturally in the browser/TypeScript ecosystem. The production system will need well-typed graph data structures, permission-safe graph queries, layout state, filters, and interaction logic. A Next.js/TypeScript architecture can define and carry those types from database query to visualization component. Python may still be useful later for specialized offline graph analysis or experiments, but it is not yet justified as a separate backend service for the MVP visualization layer.

The question I would ask is not "can FastAPI work?" It probably can. The better question is:

Does a separate FastAPI service reduce ASC's long-term complexity, or does it add another service to deploy, secure, monitor, document, debug, and hand over?

If Python was chosen partly because of team familiarity, that's a reasonable prototype rationale and worth naming as such. For production, ASC should be free to choose the architecture that best serves maintainability, governance, and long-term community use.

A reasonable alternative would be:

- use Next.js/TypeScript as the primary application stack for the MVP;
- use PostgreSQL as the system of record, with extensions such as pgvector if vector search or embeddings are part of the design;
- carry typed graph data structures and permission-safe queries from database to Three.js/SVG visualization;
- add a small Python worker or service later only if graph analysis, AI processing, or data workflows genuinely require Python;
- document API contracts, data models, workflows, permissions, prompts/system instructions, and acceptance criteria so the implementation can evolve as tooling changes.

This also connects to a broader point: in the current software environment, specific framework choices may be less durable than rich specifications, design intent, data models, prompts, system instructions, tests, and decision records. The production system

may not use the student code directly, which is fine. The most valuable outcome may be a clear design foundation that ASC or future developers can build from.

So my refinement is: FastAPI is plausible, but I do not yet see why it is necessary for ASC. The team should justify the Next.js/TypeScript plus FastAPI/Python split against a simpler Next.js/TypeScript plus PostgreSQL architecture, especially from the perspective of long-term volunteer maintainability and future graph visualization work.

(Prepared with the same AI assistance noted in the prior message. I have instigated, read, edited, and approved this note.)

Recommendations Verbose

1. Treat the volunteer admin portal as a first-class MVP feature

The member experience is important, but I think the admin experience is just as important, possibly more important for long-term success. ASC is a small, volunteer-supported organization, and the system is expected to be maintainable by volunteers or part-time administrators. The document also assumes administrators have basic digital literacy and may need onboarding.

At present, the admin requirements are mostly expressed as permissions: administrators can configure modules, import/export data, assign managers, and set visibility; managers and moderators can perform module-specific actions. That is necessary, but not sufficient.

I recommend adding a dedicated MVP section called:

Volunteer Administration and Operations Portal

This should describe task-based workflows for non-expert volunteers, including:

- Pending moderation queue.
- Expiring memberships and renewal issues.
- Failed or incomplete payments.
- Incomplete member profiles.
- Untagged or poorly tagged resources.
- Recent changes and audit-log summaries.
- Failed imports or export issues.
- Visibility and permission warnings.
- Plain-language explanations of system state.
- Safe previews, confirmations, and rollback paths for dangerous actions.

The admin portal should not feel like a raw database interface. It should help volunteers understand what needs attention, why it matters, and what action is safe to take.

Recommended requirement language:

Administrators, managers, and moderators must have role-appropriate dashboards that summarize pending work, recent changes, operational risks, and recommended next actions.

Administrative workflows must be designed for volunteers with general digital literacy, using plain-language explanations, confirmation steps, and recoverable actions where possible.

Potentially destructive administrative actions, including deletion, bulk import, permission changes, visibility changes, and data overwrite, must include preview, confirmation, audit logging, and recovery procedures where feasible.

2. Define the MVP as graph-ready, not graph-complete

A viable MVP is a reasonable first step toward graph views and a social-media-like collaboration platform. The MVP does not need to deliver the final graph/social experience, but it must create the data foundation for that future.

The current document already points in this direction. It describes the platform as supporting relationships between members, member-produced articles, publications, glossary terms, events, and other resources, with metadata tagging as the backbone for future knowledge graph visualization. The meeting notes also describe the dataset as members and resources, connected through tags and interactions such as comments, ratings, bookmarks, and “has read” activity.

However, “knowledge graph” is not a single obvious implementation. There are many possible graph models that could be built from this dataset, including a member-tag graph, resource-tag graph, member-resource interaction graph, tag co-occurrence graph, curated concept graph, activity graph, or hybrid graph. These approaches answer different questions and have different privacy, governance, and maintenance implications.

The graph may also have many dimensions. The underlying data may include member interests, resource tags, authorship, reading activity, bookmarks, ratings, comments, projects, groups, events, locations, chronology, collaboration intent, and inferred similarity. Reducing those dimensions into a 2D or 3D visualization is itself a design challenge. A graph layout is not neutral; it emphasizes some relationships, hides others, and can easily mislead users if the model and visualization choices are not explicit.

I recommend changing the MVP framing to:

The MVP includes graph-ready data structures, an explicit starting graph model, and basic relationship discovery. Advanced graph visualization is an extension.

For MVP, I recommend starting with a **member-resource-tag model**, rather than a whole-network graph. This means the system should capture members, resources, tags/concepts, and member-resource interactions in a structured way. That model can later support graph visualization, member matching, curated collections, learning journeys, and social discovery without forcing the team to build the full graph interface immediately.

The MVP should capture:

- Members.
- Resources.
- Glossary entries.
- Publications.
- Projects.
- Groups/chapters.
- Tags/concepts.
- Submissions.
- Revisions.
- Moderation status.
- Declared member interests.
- Curated resource tags.
- Member-resource interactions such as submitted, authored, bookmarked, completed/read, commented, rated, and followed.
- Visibility and privacy settings on both entities and relationships.
- Provenance for relationships, distinguishing declared, curated, inferred, and activity-based relationships.

The graph UI can come later. The relationship model should not.

The MVP should also avoid treating all relationships as equivalent. A member declaring an interest in a topic is different from a moderator tagging a resource, which is different from a member bookmarking an article, which is different from an algorithm inferring similarity. These distinctions should be preserved in the data model because they affect trust, privacy, matching, and visualization.

The graph design should also avoid treating visualization as merely a rendering problem. If more than three relationship dimensions are involved, the team will need filtering, faceting, clustering, weighting, projection, interaction design, or dimensional reduction. Each graph view should state what dimensions it includes, what it excludes, and how users should interpret it.

The documents do not specify use of a dedicated graph database. They specify PostgreSQL as the primary database. That may be entirely appropriate at ASC's scale, but the design should clarify whether the knowledge graph will be represented relationally in PostgreSQL, generated as a projection from relational data, exported for external analysis, or eventually supported by a dedicated graph database. A graph database should not be assumed necessary for MVP unless the team can explain which graph queries require it and why PostgreSQL is insufficient.

Recommended requirement language:

The MVP must define and implement a graph-ready data model based on members, resources, tags/concepts, and member-resource interactions.

The MVP must distinguish declared, curated, inferred, and activity-based relationships.

The MVP must preserve relationship provenance, including who or what created the relationship, when it was created, and whether it was user-declared, moderator-curated, system-generated, or inferred.

The MVP must store relationship data in a way that supports future knowledge graph visualization, member matching, curated collections, learning journeys, and social discovery features.

The MVP must provide basic related-member and related-resource discovery using approved tags and relationships, even if advanced graph visualization remains an extension.

The MVP must define the initial graph model before implementation, including node types, edge types, edge directionality, edge weighting, privacy rules, relationship dimensions, and the default graph view.

Each graph view must specify which relationship dimensions it includes, which dimensions it excludes, and how included dimensions are weighted, filtered, clustered, projected, or otherwise reduced for visualization.

Graph views must include plain-language explanations of what the view shows and what it omits, especially where dimensional reduction, filtering, clustering, or weighting is used.

The design must clarify whether graph data will be stored directly in PostgreSQL, generated as a projection from PostgreSQL, exported for external network analysis, or stored in a separate graph database.

Relationship data must be exportable for future network analysis, subject to privacy and permission constraints.

Advanced 2D/3D graph visualization should remain an extension until the graph model, edge semantics, privacy rules, dimensionality strategy, intended use cases, and storage/query approach are specified.

3. Add a design document

The current SRS is useful, but I do not yet see a design document. The document contains requirements, architecture notes, technology assumptions, constraints, and module descriptions, but it does not yet explain the product design, system behavior, or design intent in enough detail.

I recommend adding a separate deliverable:

Collaborative Knowledge Hub Product and Technical Design Document

It should include the following sections.

3.1 Design intent

The document should explain:

- What this system is meant to become.
- What the MVP deliberately excludes.
- How the MVP prepares for graph, social, and AI features.
- What problems the system is solving for members.
- What problems the system is solving for volunteers and administrators.

3.2 Primary user journeys

The design document should include actual user journeys. These should not be confused with learning journeys. A user journey describes how a user moves through the product to accomplish a goal.

Examples:

- Volunteer updates a member record.
- Membership manager resolves a failed renewal.
- Moderator reviews a submitted resource.
- Member submits a glossary entry.
- Member updates profile interests and finds related resources.
- Admin imports legacy publication data.
- Manager reviews an AI-generated draft.
- Admin changes visibility of a repository.
- Moderator requests revisions before publication.

3.3 Admin portal design

This should include:

- Dashboard layout.
- Queues.
- Bulk actions.
- Error handling.
- Import/export preview.
- Audit-log review.
- Role-specific views.
- Recovery flows.
- Contextual help for volunteers.

3.4 Information architecture

This should define:

- Modules.
- Navigation.
- Public areas.
- Member-only areas.
- Moderator areas.
- Manager areas.
- Administrator areas.
- How members, resources, tags, groups, journeys, and discovery relate.

3.5 Data model

This should specify:

- Entities.
- Relationships.
- Tags.
- Permissions.
- Activity records.
- Audit logs.
- Privacy fields.
- Export model for graph/network analysis.

3.6 Community safety and moderation design

This section should specify how the system handles user-generated content, harmful behavior, reporting, moderator review, enforcement actions, privacy controls, and AI-assisted moderation.

3.7 AI and agentic design

This should specify:

- Agents.
- System instructions.
- Prompt templates.
- Knowledge bases.
- Models.
- Parameters.
- Inputs and outputs.
- Human review.
- Logging and traceability.
- Evaluation.
- Failure handling.

3.8 Architecture decisions

This should document:

- Important decisions.
- Options considered.
- Rationale.
- Risks.
- Reversal cost.
- Deferred decisions.

The design document should explain not only what will be built, but why it is being built that way.

4. Document design intent and decisions from the beginning

For this project, documentation of design intent may be as important as the code. ASC will need to maintain and extend the system after the capstone team is gone.

The current document says there should be documentation from early development, and later calls for documented APIs and developer documentation. I recommend making this more specific by requiring a version-controlled decision log.

Each decision record should include:

- Problem or context.
- Options considered.
- Decision made.
- Rationale.
- Tradeoffs.
- Risks.
- Reversal cost.
- Related requirements.
- Date and author.

Examples of decisions that should be documented:

- Why the MVP is graph-ready but not graph-complete.
- Why the admin portal is prioritized.
- Why an admin-support agent is more important than a scraper.
- Why a particular database structure is used for relationship data.
- Why AI features require human review.
- Why external harvesting is deferred.
- Why certain modules are core versus extensions.
- Why a responsive web app is sufficient for MVP.
- Why import/export uses a particular format.
- Why authentication is implemented, delegated, or limited.
- Why PostgreSQL is sufficient for the MVP graph model, or why a separate graph database is justified.
- Why a particular graph model and dimensionality-reduction or visualization strategy is selected.

This would make the project easier to hand over, audit, and continue.

5. Add an international data policy and data governance model

The current document mentions GDPR, CCPA/CPRA, and U.S. production data residency, but it does not yet define an international data policy. Because ASC is an international membership organization, the design should specify how member data is stored, processed, transferred, retained, exported, deleted, and used by third-party services.

This is especially important because the project combines international members, profile data, payment status, knowledge graphing, member matching, AI assistance, and possible social features. Those features can turn ordinary membership data into sensitive relationship and behavioral data.

The policy should cover:

- Where production data is stored.
- Where backups are stored.
- Where logs are stored.
- Where exported files are stored.
- Which third-party services process member data.
- Whether AI providers receive member data.
- Whether Google Workspace, Stripe, hosting providers, email tools, analytics tools, error logging tools, or AI providers process data outside the U.S.
- Who owns the hosting and service accounts.
- Who has administrative access.
- Data retention periods.
- Member export and deletion rights.
- How deleted data is handled in backups.
- Whether graph, matching, newsletter, or AI features may use private member activity.
- Which data is public, member-only, moderator-only, manager-only, admin-only, or private.
- How international members can control visibility, matching, newsletters, and public discoverability.

Recommended requirement language:

The system must include an international data governance policy covering data residency, cross-border data transfers, third-party processors, AI providers, backups, logs, retention, deletion, export, consent, and member privacy rights.

The system must identify which services process member data, including hosting, database, backups, email, Google Workspace, payment provider, AI provider, analytics, and error logging.

The system must document where production data, backups, logs, AI prompts/outputs, and exported files are stored.

The system must define retention periods for member profiles, membership records, audit logs, moderation records, deleted content, payment references, backups, AI logs, and exported files.

The system must specify whether private member profile fields, unpublished submissions, moderation notes, payment status, and member activity can be used for graph visualization, matching, newsletters, or AI processing.

Members must have clear controls for profile visibility, matching participation, newsletter participation, and public discoverability.

6. Add a community safety, abuse handling, and moderation model

The current document treats moderation mainly as a publication workflow for knowledge hub submissions. That is not sufficient if the system will support profiles, following, comments, member matching, social discovery, public/member-only content, or future social-media-like features.

The design should specify how the system handles harmful or abusive behavior, including harassment, spam, trolling, impersonation, doxxing, unwanted contact, discriminatory content, threats, and repeated bad-faith participation.

The MVP does not need to implement a full social-platform trust-and-safety system, but it should establish the foundation: reporting, moderator review, audit logging, visibility controls, member opt-outs, and escalation paths.

Recommended requirement language:

The system must include a community safety and moderation model covering user-generated content, profiles, comments, member interactions, member matching, and public/member-only visibility.

The system must provide a reporting mechanism for members to report harmful content, abusive behavior, privacy concerns, impersonation, spam, or unwanted contact.

Moderators must be able to review reports, hide or remove content, restrict accounts, suspend access, and record enforcement actions with an audit trail.

The system must support graduated enforcement actions, such as warning, content removal, temporary restriction, suspension, and administrator escalation.

The system must preserve moderation evidence and record who took each action, when, why, and under which policy.

The system must provide privacy and safety controls for members, including profile visibility settings, opt-out from matching, and controls over whether other members may contact or follow them.

Member matching and graph discovery must not expose private information or create unwanted contact paths. Members must be able to opt out of matching and limit profile discoverability.

6.1 AI-assisted moderation

AI can help with moderation, but it should not become the final authority. This is especially important for a small scholarly community where context matters and false positives can damage trust.

AI moderation could help by flagging potentially harmful comments or profile text, detecting spam, suggesting moderation categories, summarizing report threads, identifying possible privacy leaks, detecting duplicate or low-quality submissions, and prioritizing the moderation queue.

AI moderation should not autonomously ban members, suspend accounts, delete content permanently, accuse members of misconduct, send enforcement messages, make final judgments on ambiguous cases, or override a moderator.

Recommended requirement language:

AI-assisted moderation may be used to flag, classify, summarize, or prioritize potentially harmful content, but final enforcement decisions must remain with authorized human moderators or administrators.

- AI moderation tools must provide explanations or evidence snippets for why content was flagged.
- AI moderation must be logged with model, prompt/template version, input record IDs, output, confidence or severity score, and human review outcome.
- AI moderation must be evaluated for false positives, false negatives, bias, privacy leakage, and moderator usefulness before deployment.
- The system must allow administrators to disable AI moderation without disabling the core moderation workflow.
- AI moderation must enforce the same role-based access controls and privacy restrictions as the rest of the system.

7. Replace vague “agentic” language with an actual agentic-system design

The current AI/agentic language is under-specified. The document mentions AI metadata suggestion, an AI research harvester, automated newsletter synthesis, smart member matching, and AI-powered discovery. It also says Claude will be the default AI integration. However, I do not see the design details needed to make this a credible agentic system.

If the project uses the term “agentic,” the design should specify:

- What agents exist.
- What each agent is allowed to do.
- What each agent is forbidden to do.
- System instructions.
- Prompt templates.
- Knowledge bases.
- Retrieval boundaries.
- Model selection.
- Model parameters.
- Input schemas.
- Output schemas.
- Agent relationships and handoffs.
- Human review requirements.
- Logging and traceability requirements.
- Evaluation criteria.
- Failure behavior.

Without those details, the AI portion reads more like feature brainstorming than system design.

I recommend adding a section called:

AI and Agentic System Design

For each proposed agent, the team should provide:

Field	Required content
Agent name	Clear name and purpose

MVP status	MVP, MVP-plus, or extension
Human owner	Admin, manager, moderator, or member
System instructions	Version-controlled instructions defining the agent's role, scope, constraints, tone, refusal/escalation rules, and forbidden behavior
Prompt templates	Version-controlled task prompts and output-format instructions used by the agent
Inputs	What data the agent may use
Outputs	What the agent may produce
Tools	APIs or system actions it may access
Knowledge base	Documentation, resources, profiles, logs, retrieved records, or external sources it may consult
Model	Selected model and reason
Parameters	Temperature, output limits, retrieval settings, tool-use settings, and fallback behavior
Permissions	How RBAC is enforced during retrieval, generation, and tool use
Human review	What must be approved before action
Logging and traceability	Agent version, instruction version, prompt/template version, model, parameters, input records, retrieved sources, output, review status, timestamp, and relevant cost/latency data
Evaluation	Tests and criteria for accuracy, safety, privacy, usefulness, schema compliance, and human acceptance

This distinction matters. Logging and traceability answer “what happened?” Evaluation answers “was it good, safe, and reliable?” Prompt and instruction versioning belong primarily to traceability, and they also make meaningful evaluation and debugging possible.

8. Prioritize an Administration Support Agent over a scraper

I recommend moving the external AI research harvester/scraper down the priority list. A scraper adds more content, but ASC's likely bottleneck is not lack of possible content. It is volunteer capacity: keeping member records accurate, reviewing submissions, managing renewals, handling visibility, maintaining tags, preparing updates, and avoiding operational drift.

The current document includes an AI Research Harvester for arXiv and JSTOR. That may be valuable later, but it also creates moderation, provenance, copyright, deduplication, and quality-control work.

A better first AI feature would be:

Administration Support Agent

This agent should help volunteers operate the system safely. It should be advisory and draft-oriented, not autonomous.

It should be allowed to:

- Summarize pending admin work.
- Explain member status and access issues.
- Summarize recent audit-log activity.
- Explain failed imports.
- Identify incomplete records.
- Draft renewal reminders.
- Draft moderation feedback.
- Suggest tags or cleanup actions.
- Answer questions from the admin guide.
- Link to relevant records or screens.

It should not be allowed to:

- Delete records.
- Change permissions.
- Change member status.
- Change payment status.
- Modify mailing-list membership.
- Publish content.
- Send external emails.
- Override imports.
- Expose private data beyond the current user's permissions.

Recommended requirement language:

The system should include an Administration Support Agent to assist volunteer administrators and managers with routine operational work. The agent must answer questions using system documentation and authorized system data, summarize queues and audit logs, explain permission and visibility issues, draft communications, identify records requiring attention, and suggest next actions. The agent must not autonomously modify records, permissions, payment status, publication status, visibility settings, mailing-list membership, or imported data.

The Administration Support Agent must enforce the same role-based access controls as the administrative interface.

9. Use AI for review-first assistance, not autonomous action

The document already has a good pattern: managers must be able to review and edit generated summaries before distribution. I recommend applying this rule across the AI design.

A safe general principle would be:

No AI agent may publish, email, change member data, change permissions, change payment state, change visibility, or import/overwrite records without explicit human approval.

AI is appropriate for:

- Drafting.
- Summarizing.
- Explaining.
- Suggesting.
- Classifying.
- Preparing moderation queues.

AI should require review for:

- Newsletters.
- Member matching.
- Public-facing summaries.
- External resource ingestion.
- Tag creation.
- Profile summaries.
- Anything involving private member data.
- AI-assisted moderation outputs that could affect member status, content visibility, or enforcement actions.

10. Clarify the MVP boundary

The current document contains a lot of desirable features, including events, PWA support, knowledge graphs, member matching, content synthesis, AI discovery, OJS integration, payments, Google Workspace integration, and reusable authentication. Some are marked as extensions; others still appear in ways that could confuse implementation priorities.

Given the likely scale of about 500 users, I recommend a focused MVP.

Recommended MVP

- Member profiles.
- Roles and permissions.
- Volunteer admin dashboard.
- Member management.
- Groups/chapter containers.
- Knowledge hub repositories.
- Submission and moderation workflow.
- Tags/concepts.
- Revision history.
- Audit logs.
- Search and filtering.
- Import/export with preview and validation.
- Basic relationship capture.
- Basic related-content/member discovery.
- Basic reporting and moderation workflow for harmful content or behavior.
- Privacy and safety controls for profile visibility, discoverability, and matching opt-out.
- Admin documentation.
- Design decision log.

- Optional: Administration Support Agent prototype.

MVP-plus

- Metadata Suggestion Agent.
- Curated collections.
- Newsletter drafting, human-reviewed.
- Basic graph view of one member/resource and immediate relationships.
- AI-assisted moderation prototype for flagging, summarizing, or prioritizing reports, with human review.

Post-MVP

- External research scraper.
- Advanced graph visualization.
- Social feed.
- Comments.
- OJS integration.
- Events.
- PWA/offline features.
- Full member matching.
- Automated newsletters.
- Reusable authentication server for other ASC applications.

11. Reconsider “learning journeys” and distinguish them from user journeys

The learning journeys section currently feels conceptually unclear. The idea may be useful, but it reads as if it may have been confused with user journeys. A user journey is a product-design artifact describing how a user accomplishes a goal in the system. A learning journey is a content feature: a curated path through resources.

The current learning journey section is provisional and raises unresolved questions, including whether the real requirement is simply that members can mark entries as “read.” That uncertainty suggests the feature is not ready for MVP as currently written.

I recommend either deferring learning journeys or simplifying them into:

Curated Collections

A curated collection could be a moderator-approved grouping or sequence of knowledge hub entries with:

- Title.
- Description.
- Ordered or unordered resources.
- Tags.
- Optional notes.
- Visibility setting.
- Approval workflow.
- Optional member bookmarking.

This would preserve the useful idea without requiring a course-like system, completion tracking, reflections, or learning analytics before those needs are clear.

Recommended requirement language:

The MVP may support curated collections of approved knowledge hub resources. Collections may be ordered or unordered and may include a title, description, tags, visibility setting, and curator notes.

Course-like learning journeys, completion tracking, reflections, and learning analytics should remain extensions unless their user need, governance model, and maintenance burden are clearly specified.

12. Separate the knowledge graph problem from the member matching problem

The knowledge graph and member matching are related, but they are not the same design problem. I recommend treating them as two distinct sections.

12.1 Knowledge graph: specify the graph-construction approach

There are many ways to build a knowledge graph from this dataset. The current documents mention members, resources, tags, and interactions, and they reference network-analysis research, including two-mode networks and Valdis Krebs's work. However, I do not yet see a specified graph-construction approach.

The SRS says members should be able to view “various visualisations” of connections between members, tags in profiles, and tags in Knowledge Hub entries. Elsewhere, the project describes interactive 2D/3D visualization of relationships between members and tags, dependent on member profiles and module data. That is not yet a knowledge-graph design.

The 2026-05-22 meeting notes provide a useful starting point: the dataset has two main entity types, members and resources; all entities have tags; and member-resource interactions include comments, ratings, bookmarks, and “has read.” The note says this seems suited to two-mode social analysis. That is promising, but it should be formalized into a design decision.

The graph design should specify:

- Node types.
- Edge types.
- Edge sources.
- Edge weights.
- Directionality.
- Privacy rules.
- Update strategy.
- Default graph view.
- Export format.
- What questions the graph is intended to answer.
- What research justifies the starting approach.
- Which relationship dimensions are included or excluded from each view.
- How dimensionality is reduced, filtered, clustered, weighted, or projected for visualization.
- Whether graph data is stored in PostgreSQL, generated as a projection, exported for external analysis, or supported by a dedicated graph database.

Possible graph approaches include:

Approach	What it emphasizes	Main risk
Tag co-occurrence graph	Tags connected when they appear on the same members/resources	Can be shallow or noisy
Member-tag graph	Members connected by declared interests	Easy to build, but self-reported and sparse
Resource-tag graph	Resources connected through curated tags	Useful for knowledge navigation
Member-resource graph	Members connected to resources through authorship, reading, bookmarking, rating, or commenting	Stronger evidence, but needs activity data
Resource-resource graph	Resources connected by shared tags, citations, authors, or collections	Useful, but requires curation
Curated concept graph	Moderators define meaningful concept relationships	Higher quality, more labor
Activity graph	Relationships inferred from user behavior	Useful but privacy-sensitive
Hybrid graph	Combines declared, curated, and behavioral relationships	Powerful but requires clear governance

Recommended MVP approach:

Start with a member-resource-tag model rather than a whole-network graph.

This means:

- Members have declared tags.
- Resources have curated tags.
- Members interact with resources.
- Basic graph views are generated from these relationships.
- Advanced inference comes later.

Recommended requirement language:

The MVP must define a graph-ready data model based on members, resources, tags, and member-resource interactions.

The system must distinguish declared, curated, and inferred relationships.

The system must distinguish public, member-only, moderator-only, and private relationship data.

The system must specify how high-dimensional relationship data is reduced or filtered for each graph view.

The system must explain the storage and query approach for graph data, including whether the graph is modeled inside PostgreSQL, projected from relational data, exported, or stored in a separate graph database.

Advanced graph visualization should remain an extension until the graph model, edge semantics, privacy rules, and intended use cases are specified.

12.2 Member matching: specify the matching theory

Member matching has a different issue. The problem is not graph construction; the problem is that the theory of matching is under-specified.

The SRS says smart member matching uses AI to connect members based on shared interests and skills, and that it processes profiles, publications, and activity to determine suggestions. It also says potential matches are based on keywords and concepts from member profiles and linked publications.

That is too vague. Are members being matched because they are similar, complementary, nearby, available, senior/junior, interested in the same project, or productively different?

These are different matching products.

Matching mode	Logic	Use case
Similarity matching	"You both care about X"	Find peers
Complementarity matching	"You need X; they offer X"	Find collaborators
Mentor matching	"Experienced member in your interest area"	Onboarding and guidance
Locality matching	"You are near each other or in the same time zone"	Regional groups/events
Project matching	"Your project goals overlap"	Working groups
Serendipity/adjacency matching	"Different but nearby interests"	Innovation and interdisciplinary discovery
Learning matching	"People following this pathway also used these resources"	Resource discovery

"Similar interests" and "complementary interests" are not the same requirement. If both are desired, they should be separate matching modes with separate data requirements and separate explanations to users.

For MVP, I recommend only transparent, explainable matching based on declared interests and tags.

Recommended MVP matching modes:

1. **Similar interests**
- Uses shared tags, resources, groups, and profile interests.

○ Example explanation: "You both list second-order cybernetics and organizational systems."
2. **Adjacent interests**
- Uses graph proximity or related tags.

○ Example explanation: "Your work on systems design is near this member's work on organizational cybernetics."
3. **Collaboration fit**

- Uses declared skills, project needs, collaboration interest, and availability.
- This should only be implemented if the system actually collects those fields.

Recommended requirement language:

The member matching system must define the type of match being produced, such as similarity, complementarity, mentorship, locality, project fit, or adjacency.

Each match must include a plain-language explanation based on visible, permission-appropriate data.

Members must be able to opt out of matching.

Complementarity matching must not be implemented unless the system collects explicit collaboration-intent data such as skills offered, help wanted, availability, and project interests.

AI-driven matching should remain an extension until the matching theory, data requirements, privacy rules, and evaluation criteria are specified.

13. Add acceptance criteria

Several requirements need to be more testable. For example, import/export is currently described with “[TBD] file,” and import is described as overriding current module data. That is a risky operation and should have explicit acceptance criteria.

Examples follow.

Admin dashboard

Acceptance criteria:

- A membership manager can see members expiring in the next 30 days.
- A moderator can see submissions awaiting review.
- An administrator can see failed imports and recent critical errors.
- The dashboard hides modules the user is not permitted to manage.
- The dashboard explains why each item needs attention.

Import/export

Acceptance criteria:

- Admin can preview an import before applying it.
- Invalid rows are shown with clear error messages.
- Import does not overwrite data without confirmation.
- System creates a backup or rollback point before import.
- Import action is audit-logged.
- Export format is specified.
- Export respects privacy and permissions.

Graph-ready data

Acceptance criteria:

- Each member, resource, tag, and interaction has a stable ID.
- System records who created or changed each relationship.
- Relationship data can be exported.
- Private relationships are excluded from unauthorized views.
- Declared, curated, inferred, and activity-based relationships are distinguishable.

Administration Support Agent

Acceptance criteria:

- Agent can answer questions from admin documentation.
- Agent can summarize only records the current user is authorized to view.
- Agent can draft but not send communications.
- Agent cannot change permissions, member status, visibility, payment state, or publication state.
- Every agent response is logged with source records, model, and prompt/template version.

Community safety and moderation

Acceptance criteria:

- Members can report harmful content, abusive behavior, privacy concerns, spam, impersonation, or unwanted contact.
- Moderators can review reports in a dedicated queue.
- Moderators can hide, remove, restrict, or escalate content and accounts according to documented policy.
- All moderation actions are audit-logged.
- Members can control relevant visibility and discoverability settings.
- AI-assisted moderation, if enabled, can flag or summarize issues but cannot make final enforcement decisions.

Member matching

Acceptance criteria:

- Match type is shown to the user.
- Match explanation is shown to the user.
- User can opt out.
- Private data is not exposed in explanations.
- Matching can be disabled by administrators.
- Matching logic is documented.

Knowledge graph

Acceptance criteria:

- Graph node types are documented.
- Graph edge types are documented.
- Edge sources are documented.
- Default graph view is specified.
- Privacy rules are enforced before graph data reaches the client.

- Whole-network visualization is not required for MVP.
- Each graph view specifies included and excluded relationship dimensions.
- Each graph view explains any filtering, clustering, weighting, projection, or dimensionality reduction used.
- The graph storage/query approach is documented.

14. Recommended revised MVP and extension structure

I recommend restructuring the feature priority as follows.

Core MVP

- Member profiles.
- Roles and permissions.
- Volunteer admin dashboard.
- Member management.
- Groups/chapter containers.
- Knowledge hub repositories.
- Submission and moderation workflow.
- Tags/concepts.
- Revision history.
- Audit logs.
- Search and filtering.
- Import/export with preview and validation.
- Graph-ready relationship capture.
- Basic related-content/member discovery.
- Basic reporting and moderation workflow for harmful content or behavior.
- Privacy and safety controls for profile visibility, discoverability, and matching opt-out.
- Admin documentation.
- Design decision log.

Optional MVP prototype

- Administration Support Agent.

MVP-plus

- Metadata Suggestion Agent.
- Curated collections.
- Newsletter drafting, human-reviewed.
- Basic ego-network view for one member, resource, or tag.
- AI-assisted moderation prototype for flagging, summarizing, or prioritizing reports, with human review.

Post-MVP extensions

- External research scraper.
- Advanced graph visualization.
- Social feed.

- Comments.
- OJS integration.
- Events.
- PWA/offline features.
- Full member matching.
- Automated newsletters.
- Reusable authentication server for other ASC applications.
- Course-like learning journeys.

Bottom line

I support the direction of the project, but I recommend tightening the design before implementation proceeds too far.

The strongest version of this project is not a broad AI-powered social platform from day one. It is a reliable, volunteer-operable knowledge and membership system that captures high-quality relationship data and can grow into graph-based discovery, member matching, learning journeys, and social collaboration over time.

The most important revisions are:

1. Make the volunteer admin portal a core MVP feature.
2. Define the MVP as graph-ready, not graph-complete.
3. Add a design document covering user journeys, admin workflows, data model, and AI architecture.
4. Maintain a decision log capturing design intent, tradeoffs, and architectural choices.
5. Add an international data policy and data governance model.
6. Add a community safety, abuse handling, and moderation model.
7. Replace vague agentic claims with a concrete agentic-system design.
8. Prioritize an Administration Support Agent over an external scraper.
9. Require human review for AI outputs affecting members, public content, permissions, payments, communications, or moderation outcomes.
10. Clarify the MVP boundary.
11. Rework learning journeys as either deferred features or simpler curated collections.
12. Treat knowledge graph design and member matching as two separate design problems.
13. Add testable acceptance criteria for admin operations, graph-readiness, import/export, community safety, AI behavior, matching, and graph visualization.
14. Restructure the MVP and extension priority list.